VeriText Plagiarism Platform

Phase 2 Continuous Evaluation: Technical Audit, System Architecture & In-Class Testing Guide

Project:	VeriText — Full-Stack Academic Integrity & Code Plagiarism Platform
Course Module:	Continuous Evaluation Phase 2 — Full-Stack Web Development
Cloud Backend:	https://veritext-backend-oe56.onrender.com (Render Web Service)
Cloud Frontend:	https://rt2407-08.github.io/The_Assignment_Plagiarism/ (GitHub Pages)
Database:	MongoDB Atlas Cloud Cluster (Mongoose ODM v9.11)
Repository:	rt2407-08/The_Assignment_Plagiarism (Git Monorepo)

Project Team & Work Distribution (Strict 40 / 30 / 30)

Team Member	Share	Feature Branch	Key Responsibilities & Deliverables
Aarti Thakur (Team Lead)	**40%**	feature/cloud-deployment-frontend	Express server initialization, Render port 0.0.0.0 binding, MongoDB Atlas retry logic, health APIs, live cloud frontend wiring, reactive threshold slider, monorepo configuration.
Ishita Dahiya	**30%**	feature/mvc-architecture	Monorepo structure migration, Mongoose schemas (Assignment, Submission, Student, Instructor, SimilarityReport), Bcrypt registration, JWT login routes and auth controllers.
Krrish Makkar	**30%**	feature/similarity-algorithm	3-gram character and word tokenization, Jaccard and Dice similarity math, code comment/token normalizer, Multer in-memory upload middleware, pairwise comparison matrix.

Executive Summary: *VeriText has achieved 100% completion of Continuous Evaluation Phase 2 requirements. The application features full MVC separation, cloud deployment on Render, live MongoDB Atlas data persistence, server-side N-gram similarity computation, and comprehensive interactive controls.*

1. System Architecture & MVC Request Flow

VeriText is structured strictly under the Model-View-Controller (MVC) architectural pattern to decouple business logic from presentation and database access:

Component	Folder Path	Architectural Responsibility
Model	backend/models/	Defines Mongoose data schemas, validation constraints, and database relationships (Assignment, Submission, SimilarityReport, Student, Instructor).
View	frontend/	HTML5/CSS3 client views and React dashboard (Vite) providing responsive user interfaces, modals, and dynamic threshold controls.
Controller	backend/controllers/	Processes HTTP requests, executes business rules, coordinates database queries with Mongoose models, and returns standardized JSON envelopes.
Routes	backend/routes/	Defines RESTful endpoints and connects HTTP methods (GET, POST, PATCH, DELETE) to controller functions.
Middleware	backend/middleware/	Cross-cutting operations: JWT authorization (authMiddleware.js) and in-memory Multer file parsing (uploadMiddleware.js).
Server Entry	backend/server.js	Initializes Express, enables CORS, binds host to 0.0.0.0, establishes non-blocking Atlas database connection, and exposes health routes.

End-to-End Request Pipeline:

1. **Client Action:** A student or teacher submits a form or requests a report from the browser or Postman.
2. **Route Matching:** Express evaluates server.js and delegates to the matching modular route in backend/routes/.
3. **Middleware Execution:** Any attached middleware (e.g. verifyToken or upload.single('file')) executes.
4. **Controller Handling:** The controller extracts parameters (req.params, req.body, req.query) and queries MongoDB Atlas.
5. **Algorithm Computation:** For similarity reports, generateAssignmentReport() computes 3-gram Jaccard scores in memory.
6. **Persistence & Response:** Flagged pairs are upserted into the similarityreports collection and returned as JSON.

2. N-Gram & Jaccard Similarity Algorithm

Plagiarism detection is powered by a server-side 3-gram tokenization and Jaccard similarity mathematical model implemented in backend/utils/ngramSimilarity.js:

Mathematical Formula:
Given two submission token sets A and B, Jaccard Similarity J(A, B) is:
$$J(A, B) = |A \cap B| / |A \cup B| * 100$$
Where $|A \cap B|$ is the count of shared 3-grams, and $|A \cup B| = |A| + |B| - |A \cap B|$.

Code Normalization Logic: To defeat cosmetic code obfuscation, normalizeCode() strips single-line comments (//, #) and multi-line blocks (/* */), replaces all variable string literals with "STR" and numbers with "NUM", and collapses all whitespace into single spaces.

3. In-Class Live Demonstration & Testing Data

Use these exact scenarios during your practical evaluation to demonstrate that the application handles end-to-end assignment creation, student submissions, and real-time plagiarism detection:

Scenario A: Teacher Creates an Assignment

Open frontend/instructor.html (or use Postman POST /api/assignments) and input:

```
Title: Data Structures Quiz 1
Assignment Code: DS-TEST26
Description: Implementation of linear search and time complexity analysis.
Question 1 (Code - Python): Write a function linear_search(arr, target) that returns the index of target or -1.
Question 2 (Text): Explain the best-case and worst-case time complexity of linear search.
```

Scenario B: Student 1 (Original Author) Submits Answers

Log in as Student 1 (e.g. Roll: **2410993101**, Name: **Aman Sharma**) on student.html and submit:

```
Question 1 (Python Code):
def linear_search(arr, target):
for i in range(len(arr)):
if arr[i] == target:
return i
return -1

Question 2 (Text):
The best-case time complexity of linear search is O(1) when the target element is located at the first index. The worst-case time complexity is O(n) when the element is at the end or not present.
```

Scenario C: Student 2 (Plagiarized Copy with Renamed Variables) Submits

Log in as Student 2 (e.g. Roll: **2410993102**, Name: **Rohan Gupta**) on student.html. Student 2 renames variables and adds extra comments to evade detection:

```
Question 1 (Python Code):
# Custom searching helper
def linear_search(items, key):
for index in range(len(items)):
if items[index] == key:
return index
return -1

Question 2 (Text):
Best-case time complexity of linear search is O(1) if target element is found at the first position. Worst-case complexity is O(n) when searching through the whole list to the end.
```

Scenario D: Student 3 (Unique / Non-Plagiarized) Submits

Log in as Student 3 (e.g. Roll: **2410993103**, Name: **Pooja Kaur**) on student.html:

```
Question 1 (Python Code):
def linear_search(arr, target):
index = 0
while index < len(arr):
```

```
if arr[index] == target: return index
index += 1
return None

Question 2 (Text):
Linear search inspects every element sequentially. If found immediately, step count is 1. If
absent, all n elements are inspected.
```

Expected Algorithm Output in Report:

- ***Aman Sharma vs Rohan Gupta:*** *Similarity score* ***85% - 92% (HIGH / FLAGGED)****. The code normalizer strips comments and matches the 3-gram sequence!*
- ***Aman Sharma vs Pooja Kaur:*** *Similarity score* ***20% - 35% (LOW / SAFE)****.*
- ***Threshold Slider Demo:*** *Drag the slider from 0% up to 80% on report.html. Observe Pooja's pair disappear while the Aman-Rohan pair remains prominently flagged!*

4. Postman API Demonstration Guide

The local collection VeriText_API_Testing.postman_collection.json contains pre-built requests configured with the live cloud environment (https://veritext-backend-oe56.onrender.com/api).

Step & Action	Method & URL	Expected Response Status & Key Fields
1. Health Check	GET /api/health	**200 OK** {"success": true, "database": "connected"}
2. Fetch Assignments	GET /api/assignments	**200 OK** Returns JSON array of assignments from MongoDB Atlas with code, title, and questions.
3. Lookup by Code	GET /api/assignments/code/VT-6D8RZ6	**200 OK** Returns target assignment schema for student answer view.
4. Run Server N-Gram	GET /api/assignments/:id/similarity-report	**200 OK** Returns server-computed report with totalSubmissions, flaggedPairs, and questionDetails.
5. File Upload (Multer)	POST /api/submissions (multipart/form-data)	**201 Created** Buffer extracts text from uploaded file; returns saved submission with fileName.
6. JWT Instructor Login	POST /api/instructors/login	**200 OK** Returns signed JWT token and user profile.

5. Technical Viva Questions & High-Scoring Answers

Q: Why did you choose an MVC pattern?

A: MVC enforces separation of concerns. Models define validation constraints with Mongoose; Controllers encapsulate business logic and database orchestration; and Routes map clean REST URIs. This decoupled structure allowed team members to develop authentication, algorithms, and cloud APIs concurrently without merge conflicts.

Q: How does N-gram similarity work on source code?

A: We first normalize the code by stripping single/multi-line comments, replacing variable string literals with 'STR' and numbers with 'NUM', and condensing whitespace. Then, we slice the tokens into overlapping 3-word or character grams and compute the Jaccard similarity coefficient: $|A \cap B| / |A \cup B| * 100$.

Q: Why MongoDB Atlas instead of MySQL?

A: Assignments have dynamic question schemas (varying counts of text vs code questions with custom word bounds), and similarity reports store nested comparison arrays. MongoDB's document model represents these hierarchical structures natively as JSON documents without requiring expensive relational multi-table joins.

Q: How does the Threshold Slider work?

A: In React, the slider is bound to state via useState(0). Updating the slider triggers useMemo, instantly filtering pairs where similarity >= threshold. In our HTML/JS portal, an input event listener triggers renderFilteredPairs() to update table rows in real-time without reloading the page.

Q: How is file upload handled securely with Multer?

A: We use multer.memoryStorage() so files are buffered in RAM (req.file.buffer) rather than saved to disk. We enforce an allowed extension whitelist (.py, .java, .cpp, .txt) and extract UTF-8 text directly into the submission content, eliminating disk storage leaks and script execution risks.